SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192521O62
Name	KEERTHANA.L

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 25**
Question Count: 5

Code of Conduct

I _NANDHINI SRI V(192521344)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: _____

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

ANSWER:

Hadoop Job Fails Due to Uneven Input Splits

Problem

A Hadoop MapReduce job repeatedly fails because the input data is divided into uneven splits. Some reducers receive a very large amount of intermediate data, while other reducers complete their tasks quickly and remain idle. This results in poor resource utilization and increases the overall execution time.

Likely Design Error

The main design error is **data skew and improper partitioning of input data**. Hadoop divides input files into blocks and processes them using mapper tasks. If the data is not distributed evenly, certain mapper or reducer tasks become overloaded.

For example, if one key occurs much more frequently than other keys, the corresponding reducer may receive a huge amount of data while other reducers receive very little.

Corrective Actions

- Configure an appropriate **HDFS block size**.
- Divide large datasets into reasonably balanced input files.
- Use a suitable **partitioner** to distribute keys evenly among reducers.
- Identify and handle **data-skewed keys**.
- Use **combiners** to reduce the amount of intermediate data transferred to reducers.
- Use salting techniques for highly frequent keys.
- Monitor mapper and reducer execution times.
- Repartition the input data when necessary.
- Avoid creating excessive numbers of very small files.

Expected Result

After balancing the input and distributing keys properly, mapper and reducer tasks will have more uniform workloads. This improves parallel processing, reduces execution time, and makes better use of cluster resources.

Conclusion

The Hadoop job failure is mainly caused by **uneven data distribution and data skew**. Proper input splitting, partitioning, combiners, and skew-handling techniques can prevent reducer idling and improve overall Hadoop performance.

2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

ANSWER:

HDFS Data Unavailability After Single Node Failure

Problem

An HDFS deployment experiences frequent data unavailability whenever a single DataNode fails. Since HDFS is designed to provide fault tolerance, a single node failure should normally not cause permanent data unavailability.

Probable Design Error

The most likely problem is an **inadequate replication factor**. If an HDFS block has only one copy and the DataNode containing that block fails, the block becomes unavailable.

Another possible problem is improper **NameNode architecture**. If only one NameNode is being used and it fails, the HDFS namespace may become unavailable even though the DataNodes are still operating.

Corrective Actions

- Configure an appropriate **HDFS replication factor**, such as 3 for important data.
- Ensure replicas are stored on different DataNodes.
- Use **rack-aware replication** so that replicas are not located in the same physical rack.
- Monitor under-replicated and missing blocks.
- Allow HDFS to automatically re-replicate blocks after a node failure.
- Implement **NameNode High Availability (HA)**.
- Use Active and Standby NameNodes.

- Use JournalNodes or an appropriate shared-edit mechanism for maintaining NameNode state.
- Regularly test DataNode and NameNode failure recovery.

Expected Result

If one DataNode fails, another DataNode containing a replica can immediately provide the required block. Similarly, NameNode HA allows the Standby NameNode to take over if the Active NameNode fails.

Conclusion

The problem is most likely caused by **insufficient block replication or a single point of failure in the NameNode**. Proper replication, rack awareness, and NameNode HA provide fault tolerance and improve HDFS availability.

3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

ANSWER:

Problem

An organization uses an ETL workflow with Apache NiFi to transfer data into an analytics store. However, the same records are appearing multiple times, resulting in incorrect analysis and unreliable reports.

Likely Causes

Duplicate records can occur for several reasons:

- The same FlowFile may be processed more than once.
- A failed transaction may be automatically retried.
- The source system may send the same record multiple times.
- Multiple NiFi flows may process the same source data.
- There may be no unique identifier for each record.
- State or checkpoint management may be incorrectly configured.
- The destination database may not have uniqueness constraints.

Recommended Fixes

- Assign a **unique ID** to every transaction or event.
- Use NiFi **state management** for tracking already processed data.
- Maintain source checkpoints or timestamps.
- Configure processors properly to prevent unnecessary reprocessing.
- Implement **idempotent processing**, where processing the same record again does not create another copy.
- Apply deduplication before loading data into the analytics system.

- Use database primary keys or unique constraints.
- Monitor failed, queued, and retried FlowFiles.
- Maintain proper logging and audit information.

Example

Suppose a transaction has the ID TX1001. If NiFi receives TX1001 again, the system should check whether that ID already exists. If it exists, the duplicate should be rejected or ignored instead of inserting another record.

Expected Result

With unique IDs, state tracking, deduplication, and idempotent processing, the analytics store will contain one valid copy of each transaction.

Conclusion

The duplicate-record problem is generally caused by **reprocessing, missing unique identifiers, or incorrect state management**. Proper NiFi configuration combined with destination-side uniqueness controls can provide reliable and duplicate-free data ingestion.

4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

ANSWER:

YARN Cluster Suffers from Resource Starvation

Problem

A YARN cluster is shared by multiple users. When several users submit jobs at the same time, some applications receive insufficient CPU and memory resources. As a result, jobs remain pending for a long time and the cluster suffers from resource starvation.

Likely Design Error

The main design error is **poor resource scheduling and lack of proper queue management**. If one user or application consumes most of the available cluster resources, other applications cannot receive sufficient resources.

Incorrect configuration of memory, CPU, application limits, and scheduler policies can also contribute to the problem.

Better Resource Management Approach

The organization should use **YARN Capacity Scheduler or Fair Scheduler**.

Capacity Scheduler

The cluster can be divided into multiple queues:

- **Production Queue – 40%**
- **Analytics Queue – 35%**
- **Research Queue – 25%**

Each queue receives a guaranteed share of cluster resources.

Additional Corrective Actions

- Configure CPU and memory resource limits.
- Set maximum resources that a single application can consume.
- Configure minimum and maximum resources for queues.
- Set user-level resource limits.
- Use queue priorities when necessary.
- Monitor ResourceManager and NodeManager performance.
- Prevent one large job from consuming the entire cluster.
- Use fair scheduling when workloads from different users should receive balanced resources.
- Regularly review resource utilization and adjust queue capacities.

Expected Result

Each user or department receives a reasonable share of cluster resources. Large jobs can continue running without completely blocking smaller jobs.

Conclusion

The resource starvation problem is mainly caused by **improper YARN scheduling and resource allocation**. Capacity/Fair scheduling, queue management, CPU-memory limits, and monitoring can improve cluster utilization and provide fair resource access.

5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

ANSWER:

Outdated Data After Distributed Storage Recovery

Problem

A distributed storage system experiences a failure. During recovery, the backup system restores an older version of the data instead of the latest valid version. This results in data loss and inconsistency.

Probable Design Flaw

The major design flaw is **reliance on stale backups or poorly managed replication/versioning**. Replication creates multiple copies of data, but if incorrect or deleted data is replicated, those changes may also be propagated to replicas.

Another issue may be that backups are performed too infrequently. For example, if a backup is created once every 24 hours, transactions created after the last backup may be lost during recovery.

Corrective Actions

- Perform regular full and incremental backups.
- Use **backup versioning** to maintain multiple recovery points.
- Use snapshots where supported.
- Maintain timestamps and version numbers.
- Define a suitable **Recovery Point Objective (RPO)**.
- Define a suitable **Recovery Time Objective (RTO)**.
- Store backups separately from the primary storage system.
- Protect backup copies from accidental deletion or corruption.
- Test restoration procedures regularly.
- Monitor backup completion and failure status.
- Use replication together with backup rather than treating replication as the only backup mechanism.

Example

If the last backup was created at 12 PM and the storage system fails at 6 PM, restoring that backup may lose six hours of transactions. More frequent incremental backups can reduce this loss.

Expected Result

With frequent backups and multiple recovery points, administrators can select the **latest valid version** of the data instead of restoring an outdated copy.

Conclusion

The outdated-data problem is caused by **insufficient backup frequency, poor version management, or incorrect replication design**. A combination of incremental backups, versioning, snapshots, replication, and regular recovery testing provides reliable and up-to-date data recovery.